

Report for Mandatory Assignment 4 (Mutual Exclusion)

Group name: JAA

Members(abch, asjo, jnov)

11/11-2024

- **Describe how your system meets the System Requirements (R1, R2, and R3):**

- R1: The system is designed as a peer-to-peer architecture, using a set of peer nodes, each Node acts as both a client and a server. The system simulates the token ring algorithm, where a token is passed around the nodes in a ring-like structure. Each Node is implemented as a Peer Struct, which has several fields including an ID and a reference to the next Node in the ring. Any Node can at a random time interval request access to the Critical Section. If a Node has requested access to the Critical Section and is granted the token, it will enter the Critical Section, before passing on the token. The Critical Section is emulated by logging print statements, which display when the node enters and exits the section.

R2 (Safety): The system ensures exclusive access to the Critical Section by allowing only one node to hold the token at any given time. Each Node has a channel(*token_chan*) which is used to receive the token. Each Node has a client which holds an established connection to the next Node server. This enables the Nodes to use GRPC to communicate and pass on the token. When the algorithm starts Node 0 will start with the token, then pass it to Node 1, Node 1 then passes it to Node 2, and so on. When a node does not have the token it will always stand by and be ready to receive a token on its channel. If a Node has a Token it will either enter the critical section or pass it on to the next Node in the ring.

R3 (Liveliness): The system uses a token ring structure, where each node is assigned an ID and connects to the next node in the sequence based on their IDs. With this ring structure, the token circulates from one node to the next, ensuring that any node that has requested access to the Critical Section, will eventually gain access. It is important to note that we here assume that no nodes fail during execution. If a node fails, the token will not be passed on.

- **Discuss the used algorithm**

The algorithm implements a token ring protocol for mutual exclusion, using a token that circulates among nodes in a ring to grant access to a critical section. In this

implementation, three key functions manage access to the critical section and token passing.

- **EnterCs_andPasstoken:** When the program starts, each node initiates EnterCs_andPasstoken() in a separate goroutine, entering an infinite loop to await the token via their channel. When a node receives the token, it checks its **request** flag to determine if it needs access to the critical section. If the flag is **true**, the node enters the critical section, simulates work, and then resets the flag upon exit. Regardless of critical section access, the node then passes the token to the next node by calling the gRPC function ReceiveToken().
- **ReceiveToken:** This function is called by a Node's predecessor. When a Node is executing EnterCs_andPasstoken() and has finished its critical section work or decided to skip it, it uses its connection to the next Node in the ring, to call ReceiveToken() on that next Node. ReceiveToken() places the received token into the Node's channel. This will make EnterCs_andPasstoken() resume as it now has received a token, and then it processes the token and decides what to do. For example:

Node 0 executes EnterCs_andPasstoken(), decides it doesn't need the token, and calls ReceiveToken() on **Node 1**.

Node 1's ReceiveToken() method places the token in the channel for Node 1.

Node 1 will now execute EnterCs_andPasstoken(), which has been blocked by its channel waiting for a token, now it unblocks because there's a value in the channel. Now Node 1 will call ReceiveToken() on **Node 2**, and so on.

This chain of calls enables the token to circulate the ring. Below are the logs from executing the algorithm

```
To start the simulation write : 'START' and hit
START
Node 0 got encrypted token
Node 1 got encrypted token
Node 2 got encrypted token
Node 2 is requesting access critical section
Node 0 got encrypted token
Node 1 got encrypted token
Node 1 is requesting access critical section
Node 0 is requesting access critical section
Node 2 got encrypted token
Node 2 is entering critical section
Node 2 is leaving critical section
Node 0 got encrypted token
Node 0 is entering critical section
Node 0 is leaving critical section
Node 1 got encrypted token
Node 1 is entering critical section
Node 1 is leaving critical section
Node 2 got encrypted token
```

- **SimulateRequest:** Each node also initiates SimulateRequest() in a separate goroutine. This function will continuously ask for access to the critical section. If the **request** flag is set to **false**, the function will set the node's **request** flag to **true** at random intervals, indicating a request for critical section access. This ensures that a Node can ask for access at all times, and helps control when a Node can enter the critical section. This method logs a request message to the log, as seen in the snippets provided.

- **System logs of the program executing until manually aborted:**

```
Node 1 has connected to node 2
Node 0 has connected to node 1
Node 2 has connected to node 0

Hello and welcome to a token ring simulation that demonstrates distributed mutual exclusion for three Nodes
To start the simulation write : 'START' and hit enter. To quit the program hit ctrl+c
START
Node 0 got encrypted token
Node 1 got encrypted token
Node 2 got encrypted token
Node 2 is requesting access critical section
Node 0 got encrypted token
Node 1 got encrypted token
Node 1 is requesting access critical section
Node 0 is requesting access critical section
Node 2 got encrypted token
Node 2 is entering critical section
Node 2 is leaving critical section
Node 0 got encrypted token
Node 0 is entering critical section
Node 0 is leaving critical section
Node 1 got encrypted token
Node 1 is entering critical section
Node 1 is leaving critical section
Node 2 got encrypted token
Node 0 is requesting access critical section
Node 2 is requesting access critical section
Node 0 got encrypted token
Node 0 is entering critical section
Node 0 is leaving critical section
Node 1 got encrypted token
Node 2 got encrypted token
Node 2 is entering critical section
Node 1 is requesting access critical section
Node 2 is leaving critical section
exit status 0xc000013a
```

- **Link to Git repository:**

https://github.com/Asger124/Distributed/tree/Asger_tcp/Mutual_Exclusion